Principios SOLID

¿Qué son los principios SOLID?

- **Basado en 5 principios que tienen como objetivo desarrollar aplicaciones basadas en arquitecturas limpias, robustas, legibles y escalables:**
  - ***S*ingle Responsability**
  - ***O*pen/Close**
  - ***L*iskov Substitution**
  - ***I*nterface Segregation**
  - ***D*ependency Inversion**

Principio Single Responsability

- **Según este principio, una clase debe tener una única responsabilidad. No debe mezclar métodos que realicen tareas no relacionadas.**
- **La siguiente clase por ejemplo, no cumple el principio:**

```
public class Empleado {
    private String nombre;
    private double salarioBase;
    //constructores getter and setter
    public double calcularSalario() {
        return salarioBase * 1.2;
    }
    public void guardarEnFichero() {
        try (FileWriter writer = new FileWriter(nombre + ".txt")) {
            writer.write("Nombre: " + nombre + ", Salario: " +
calcularSalario());
        } catch (IOException e) {
            e.printStackTrace();
        }
     }
    }
```

Este método no tendría que estar en esta clase

Según este principio, una clase que se dedica a una determinada cosa o aspecto, tendrá solo un único motivo para cambiar.

Principio Single Responsability

- **La solución correcta sería dividirlo en dos clases:**

```
//clase dedicada a empleados y calculos sobre el mismo
public class Empleado {
    private String nombre;
    private double salarioBase;
   //constructores, getter and setter
    public double calcularSalario() {
        return salarioBase * 1.2;
    }
}
```

Podría tener otros métodos relacionados con el empleado, como aumentarSalario(), etc.

```
// Clase dedicada a persistir datos
public class EmpleadoFileWriter {
    public void guardar(Empleado empleado) {
        try (FileWriter writer = new
FileWriter(empleado.getNombre() + ".txt")) {
            writer.write("Nombre: " +
empleado.getNombre() + ", Salario: " +
empleado.calcularSalario());
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Principio Single Responsability

- **Ventajas:**
  - ***Permite aislar los cambios*, lo que significa que si cambia la lógica de negocio o la forma de persistir datos, solo una clase se ve afectada**
  - ***Facilita las pruebas*. Al tratarse de clases más pequeñas y con un único propósito son más fáciles de probar.**
  - **Reutilización código. Dado que están bien delimitadas, pueden usarse en diferentes contextos.**
  - **Reduce el acoplamiento. Las clase dependen en menor medida del resto de clases**

Principio Open/Close

- Según este principio, una clase debe estar abierta para su extensión, pero cerrada para su modificación.

- Esto significa que debe estar preparada para ampliar su funcionalidad, sin que implique modificar los métodos de la clase.

- La siguiente clase no cumple con el principio:

Cada vez que se incluyera un nuevo tipo de notificación, habría que cambiar el código

```
class NotificationService {
    public void sendNotification(String type, String message) {
        if (type.equals("EMAIL")) {
            System.out.println("Enviando email: " + message);
        } else if (type.equals("SMS")) {
            System.out.println("Enviando SMS: " + message);
        }
    }
}
```

Principio Open/Close

- **Para cumplir con el principio, la clase debería disponer de un único método que sirviera para cualquier objeto.**
- **Se define una interfaz con el método que sería implementada por los tipos de objetos.**

```
interface Notification {
    void send(String message);
}
// Notificación por email
class EmailNotification implements Notification {
    @Override
    public void send(String message) {
        System.out.println("Enviando email: " + message);
    }
}
// Notificación por SMS
class SmsNotification implements Notification {
    @Override
    public void send(String message) {
        System.out.println("Enviando SMS: " + message);
    }
}
```

Mediante polimorfismo, la clase trabajaría contra la interfaz y, por tanto, "abierta" a cualquier tipo de objeto que la implemente

```
// Servicio de notificaciones
class NotificationService {
    public void sendNotification(Notification notification,
            String message) {
      // No hay que modificar el servicio
      notification.send(message);
    }
}
```

Principio de sustitución de Liskov

- Este principio dice que, dada una superclase, cualquier código donde se haga uso de esta clase debería funcionar si se sustituye un objeto de esa clase por otro de cualquiera de las subclases.
- Supongamos lo siguiente:

```
abstract class Figura{
      public abstract double area();
      public abstract int perimetro();
}
class Rectangulo extends Figura{
      int alto,ancho;
      public double area(){return ancho*alto;}
      public int perimetro(){return 2*ancho+2*alto;
}
class Esfera extends Figura{
      int radio;
      public double area(){return 4*Math.PI*radio*radio;}
      //una esfera no tiene perímetro!!
      public int perimetro(){
            throw new UnsuportedOperationException();
      }
}
```

Principio de sustitución de Liskov

- Si en otra clase tenemos el siguiente método:

```
mostrarPerimetro(Figura f){
    System.out.println(f.perimetro());
}
```

- Fallará al llamarlo con un objeto Esfera. Luego esa clase no cumple con el LSP
- Lo correcto sería separar perimetro() de Figura, ya que no todas las figuras tienen perímetro:

```
abstract class Figura{
    public abstract double area();
}
interface Perimetrable{
    int perimetro();
}
```

Principio de sustitución de Liskov

- **Las subclases Figura quedarían:**

```
class Rectangulo extends Figura implements Perimetrable{
      int alto,ancho;
      public double area(){return ancho*alto;}
      public int perimetro(){return 2*ancho+2*alto;
}
class Esfera extends Figura{
      int radio;
      public double area(){return 4*Math.PI*radio*radio;}
}
```

- **Y para manejar perímetros y figuras:**

```
mostrarPerimetro(Perimetrable p){
     System.out.println(p.perimetro());
}
procesarFiguras(Figura f){
     :
}
```

Principio de Segregación de interfaz

- El principio dice que una clase no debe ser obligada a implementar métodos que no va a utilizar o no le corresponden. Esto ocurre normalmente cuando definimos interfaces con muchos métodos.
- Por ejemplo:

```
interface Listener{
    void close();
    void click();
    void mouseEnter();
}
```

- Esta clase viola el principio porque obligaría a las clases a implementar métodos que no le corresponden

Principio de Segregación de interfaz

```
classs Window implements Listener{


}
```

```
class Button implements Listener{


}
```

- **Estas clases violan el principio, pues Window no tiene porque implementar *click()*, ni Button *close()*.**
- **La solución es ajustar las interfaces al primer principio, y que solo incluyan métodos específicos de una responsabilidad**

Principio Liskov vs Segregación

- En muchas ocasiones estos principios se confunden, ya que una segregación de interfaz mal hecha puede llevar a incumplir el principio de Liskov.
- En la aplicación del principio de segregación debemos preocuparnos de realizar interfaces simples y específicas.
- En la aplicación del principio de Liskov, a la hora de heredar o implementar una clase/interfaz debemos asegurarnos de que todos los métodos son coherentes para esa clase

EJEMPLO ISP-LSP

- **No toda interfaz grande incumple ISP. La siguiente interfaz es coherente con ISP:**

```
// Interfaz que tiene varios métodos, pero todos tienen sentido para el grupo
public interface MediaPlayer {
    void play();
    void pause();
    void stop();
    void seek(int seconds);
}
```

- **Mala aplicación de LSP**

```
public class LiveStreamPlayer implements MediaPlayer {
    @Override public void play()  { System.out.println("Live: play"); }
    @Override public void pause() { System.out.println("Live: pause"); }
    @Override public void stop()  { System.out.println("Live: stop"); }
    @Override
    public void seek(int seconds) {
        // no tiene sentido para streams en directo
        throw new UnsupportedOperationException("No se puede buscar en un stream en vivo");
    }
}
```

Principio de Inversión de Dependencia

- Según este principio, los métodos de una clase que realizan una determinada operación, por ejemplo acceder a una base de datos, no deben depender de un tipo concreto, sino de una abstracción.
- Las implementaciones concretas de estas abstracciones se inyectan en los métodos
- El siguiente ejemplo no sigue el principio, por depender de una base de datos concreta:

```
public class MySQLDatabase {
    public void save(String datos) {
        System.out.println("Guardando en MySQL: " + datos);
    }
}
```

```
public class ServicioDatos {
    private MySQLDatabase db = new MySQLDatabase();
    public void procesar(String datos) {
        //depende de la implementación para MySQL
        db.save(datos);
    }
}
```

Principio de Inversión de Dependencia

- **Para cumplir con el principio, la operación se define en una interfaz y el método de la clase de servicio opera con la implementación de esa interfaz que le sea inyectada.**

```
// Abstracción
public interface Repositorio {
    void save(String datos);
}
// Implementación concreta
public class MySQLDatabase implements Repositorio {
    public void save(String datos) {
        System.out.println("Guardando en MySQL: " + datos);
    }
}
```

```
// Clase de servicio que utiliza la abstracción
public class Servicio {
    private final Repositorio repo;
    public Servicio(Repositorio repo) {
        this.repo = repo;
    }
    public void procesar(String datos) {
        repo.guardar(datos);
    }
}
```

Principio de Inversión de Dependencia

- Si comparamos este principio con el de abierto/cerrado, vemos que hay similitudes entre ambos, aunque con objetivos diferentes.
  - OCP busca poder añadir nuevas funcionalidades sin modificar el código, para lo cual recurre a polimorfismo con interfaces
  - DIP busca reducir el acoplamiento entre módulos, para lo cual hace que los módulos dependan de interfaces y se aplique igualmente el polimorfismo para lograrlo